Give Your AI Agent a Nervous System

Mode 2 Phase 2 (Build Workers), Chapter 3/3 — Revision & Viva-Prep Cheatsheet (English + Roman Urdu)
Source: agentfactory.panaversity.org/docs/ai-agent-nervous-system-crash-course

EN: An agent already thinks and acts -- what's missing is a nervous system that closes the loop without a human: waking on events, reacting by reflex, holding its place while waiting for a human. Setup: Inngest engine (you don't write retries/waits/memory) + your agent (you do write).
Roman Urdu: Agent already sochta/act karta -- missing cheez nervous system hai jo bina insaan loop khud band karta: event par jagna, reflex se react karna, insaan ka wait karte hue apni jagah rakhna. Setup: Inngest engine (retries/waits/memory nahi likhte) + aapka agent (likhte ho).

1. The Senses -- Events, Cron, Webhooks, Idempotency, Fan-Out

- Request = sync (caller waits); Event = async (fire-and-forget, zero/one/many functions react). Events give durability/scale almost for free.
- **Idempotency, 2 layers:** stable event id, and step-level memoization (completed steps return from memo on crash, never re-run).
- Fan-out: many functions subscribe to one event, or a parent fires N child events -- sub-agent delegation is a special case of fan-out.

2. The Reflexes -- step.run, Memoization, Sleep/Wait, Retries

- step.run is a checkpoint -- each step must be re-run-safe. **Trap:** code OUTSIDE step.run re-runs on every replay -- wrap expensive work inside its own step.
- step.sleep/wait_for_event give durability through time/events with ZERO compute while waiting. ~4 retries with exponential backoff; NonRetriableError for permanent failures.
- Python note: step.ai.wrap is TypeScript-only -- in Python, always wrap the whole agent run in ctx.step.run (a retry re-runs the WHOLE agent, re-spending tokens).

3. Balance + Recovery -- Concurrency, Priority, HITL

- Concurrency = in-flight cap; Throttle = start-rate cap. Per-key concurrency stops one customer from blocking everyone else.
- Replay starts a NEW run from the top (no memo) -- prevent duplicate side-effects with idempotency keys, not memoization.
- **HITL gates** via step.wait_for_event (e.g. a $500 refund approval) -- a suspended function doesn't get billed while waiting.

Key Terms Cheatsheet

One line each — English + Roman Urdu.

Term	English	Roman Urdu
Event vs Request	Event = async, fire-and-forget, many reactors possible; Request = sync, caller waits.	Event = async, fire-and-forget, kai reactors mumkin; Request = sync, caller wait karta.
step.run	A checkpoint operation that must be re-run-safe.	Checkpoint operation jo re-run-safe honi chahiye.
Memoization	Completed steps return from memo on crash/replay instead of re-running.	Completed steps crash/replay par memo se return hote, dobara nahi chalte.
Idempotency (2 layers)	Stable event ID + step-level memoization, both needed against at-least-once delivery.	Stable event ID + step-level memoization, dono at-least-once delivery ke khilaf zaroori.
HITL gate	A durable wait for human approval, not billed while suspended.	Human approval ke liye durable wait, suspended hone par bill nahi hota.

Quick Self-Check

Question	Answer
Why does code placed OUTSIDE step.run cause bugs on replay?	It re-runs every single replay, unlike step.run's checkpointed code -- expensive or side-effecting work must be wrapped inside its own step.
Why does a Python agent retry re-spend all its tokens, unlike the TypeScript step.ai.wrap pattern?	step.ai.wrap doesn't exist for Python -- the whole agent run must be wrapped in ctx.step.run, so a retry re-runs the entire agent call.
What's the difference between an automatic retry and a Replay?	A retry preserves memoized completed steps; a Replay starts a brand-new run from the top with no memo, risking duplicate side-effects unless guarded by idempotency keys.
Why isn't a suspended HITL-gated function billed while waiting?	Its state is paged out to a durable store rather than holding compute -- waiting costs nothing until the event or timeout arrives.

Where This Leads / Sources

Leads to: OpenClaw (per-human delegate), Paperclip (manager), Claude Managed Agents (policy-expandable). Full worked example (Customer Support Worker, 7 decisions) in Part 4.

Source: docs/ai-agent-nervous-system-crash-course/ (this repo), content originally via Zia Tutor AI connector.